

MATHDNN_HW9

December 13, 2021

Problem 2

```
[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torchvision.utils import save_image, make_grid

import numpy as np
import matplotlib.pyplot as plt
```

```
[ ]: batch_size = 128
(full_dim, mid_dim, hidden) = (1 * 28 * 28, 1000, 5)
lr = 1e-3
epochs = 100
device = torch.device("cpu")
```

```
[ ]: #####
# STEP 1: Define dataset and preprocessing #
#####

class Logistic(torch.distributions.Distribution):
    def __init__(self):
        super(Logistic, self).__init__()

    def log_prob(self, x):
        return -(F.softplus(x) + F.softplus(-x))

    def sample(self, size):
        z = torch.distributions.Uniform(0., 1.).sample(size).to(device)
        return torch.log(z) - torch.log(1. - z)
```

```
[ ]: #####
# STEP 3: Implement Coupling Layer #
#####
```

```

class Coupling(nn.Module):
    def __init__(self, in_out_dim, mid_dim, hidden, mask_config):
        super(Coupling, self).__init__()
        self.mask_config = mask_config

        self.in_block = nn.Sequential(nn.Linear(in_out_dim//2, mid_dim), nn.
↪ReLU())
        self.mid_block = nn.ModuleList([nn.Sequential(nn.Linear(mid_dim,
↪mid_dim), nn.ReLU())
                                         for _ in
↪range(hidden - 1)])
        self.out_block = nn.Linear(mid_dim, in_out_dim//2)

    def forward(self, x, reverse=False):
        [B, W] = list(x.size())
        x = x.reshape((B, W//2, 2))
        if self.mask_config:
            on, off = x[:, :, 0], x[:, :, 1]
        else:
            off, on = x[:, :, 0], x[:, :, 1]

        off_ = self.in_block(off)
        for i in range(len(self.mid_block)):
            off_ = self.mid_block[i](off_)
        shift = self.out_block(off_)

        if reverse:
            on = on - shift
        else:
            on = on + shift

        if self.mask_config:
            x = torch.stack((on, off), dim=2)
        else:
            x = torch.stack((off, on), dim=2)
        return x.reshape((B, W))

class Scaling(nn.Module):
    def __init__(self, dim):
        super(Scaling, self).__init__()
        self.scale = nn.Parameter(torch.zeros((1, dim)), requires_grad=True)

    def forward(self, x, reverse=False):
        log_det_J = torch.sum(self.scale)
        if reverse:
            x = x * torch.exp(-self.scale)
        else:

```

```

        x = x * torch.exp(self.scale)
    return x, log_det_J

```

```

[ ]: #####
# STEP 4: Implement NICE #
#####

class NICE(nn.Module):
    def __init__(self, in_out_dim, mid_dim, hidden, mask_config=1.0, coupling=4):
        super(NICE, self).__init__()
        self.prior = Logistic()
        self.in_out_dim = in_out_dim

        self.coupling = nn.ModuleList([
            Coupling(in_out_dim=in_out_dim,
                    mid_dim=mid_dim,
                    hidden=hidden,
                    mask_config=(mask_config+i)%2) \
            for i in range(coupling)])

        self.scaling = Scaling(in_out_dim)

    def g(self, z):
        x, _ = self.scaling(z, reverse=True)
        for i in reversed(range(len(self.coupling))):
            x = self.coupling[i](x, reverse=True)
        return x

    def f(self, x):
        for i in range(len(self.coupling)):
            x = self.coupling[i](x)
        z, log_det_J = self.scaling(x)
        return z, log_det_J

    def log_prob(self, x):
        z, log_det_J = self.f(x)
        log_ll = torch.sum(self.prior.log_prob(z), dim=1)
        return log_ll + log_det_J

    def sample(self, size):
        z = self.prior.sample((size, self.in_out_dim)).to(device)
        return self.g(z)

    def forward(self, x):
        return self.log_prob(x)

```

```

[ ]: # Load pre-trained NICE model onto CPU
model = NICE(in_out_dim=784, mid_dim=1000, hidden=5).to(device)
model.load_state_dict(torch.load('nice.pt',map_location=torch.device('cpu'))))

# Since we do not update model, set requires_grad = False
model.requires_grad_(False)

# Get an MNIST image
testset = torchvision.datasets.MNIST(root='./', train=False, download=True,
    ↪transform=torchvision.transforms.ToTensor())
test_loader = torch.utils.data.DataLoader(testset, batch_size=1, shuffle=False)
pass_count = 6
itr = iter(test_loader)
for _ in range(pass_count+1):
    image,_ = itr.next()

plt.figure(figsize = (4,4))
plt.title('Original Image')
plt.imshow(make_grid(image.squeeze().detach()).permute(1,2,0))
# plt.show()
plt.savefig('plt1.png')

# Create mask
mask = torch.ones_like(image,dtype=torch.bool)
mask[:, :, 5:12, 5:20] = 0

# Partially corrupt the image
image[mask.logical_not()] = torch.ones_like(image[mask.logical_not()])
plt.figure(figsize = (4,4))
plt.title('Corrupted Image')
plt.imshow(make_grid(image.squeeze()).permute(1,2,0))
# plt.show()

plt.savefig('plt2.png')

# implement this
lr = 1e-3
recon = image.clone().requires_grad_(True)

# no optimiser, due to Nonetype grad issue.

for i in range(300):
    recon = recon.view(-1, 784)
    loss = -model(recon).mean()
    if(i%30 == 0):
        print("iteration ", i, ", loss = ", loss)

```

```

# preventing Nonetype grad issue
recon.retain_grad()

loss.backward()

recon_grad = recon.grad

# implement GD
with torch.no_grad():
    recon -= lr * recon_grad

recon = recon.view(-1, 1, 28, 28)
image = image.view(-1, 1, 28, 28)

with torch.no_grad():
    # bounded
    torch.clamp(recon, min = 0.0, max = 1.0)
    # projection
    recon[mask] = image[mask]

recon = recon.view(-1, 1, 28, 28)

# Plot reconstruction
plt.figure(figsize = (4,4))
plt.title('Reconstruction')
plt.imshow(make_grid(recon.squeeze().detach()).permute(1,2,0))
# plt.show()
plt.savefig('plt3.png')

```

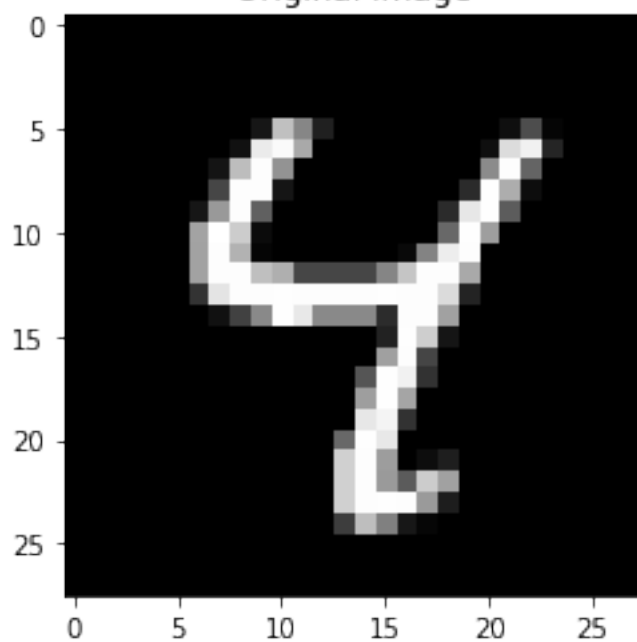
```

iteration 0 , loss = tensor(-937.8738, grad_fn=<NegBackward>)
iteration 30 , loss = tensor(-2001.9513, grad_fn=<NegBackward>)
iteration 60 , loss = tensor(-2217.0586, grad_fn=<NegBackward>)
iteration 90 , loss = tensor(-1363.6204, grad_fn=<NegBackward>)
iteration 120 , loss = tensor(-2113.6763, grad_fn=<NegBackward>)
iteration 150 , loss = tensor(-1860.3479, grad_fn=<NegBackward>)
iteration 180 , loss = tensor(-2075.3894, grad_fn=<NegBackward>)
iteration 210 , loss = tensor(-1956.3916, grad_fn=<NegBackward>)
iteration 240 , loss = tensor(-2266.7769, grad_fn=<NegBackward>)
iteration 270 , loss = tensor(-1989.3494, grad_fn=<NegBackward>)

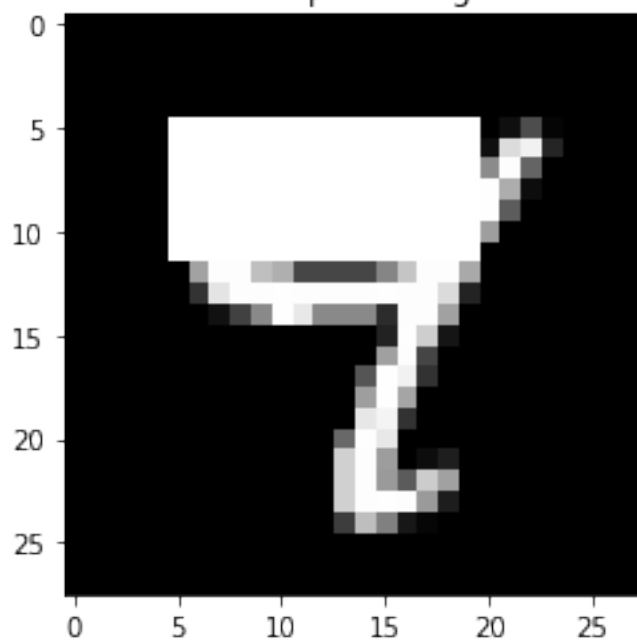
```

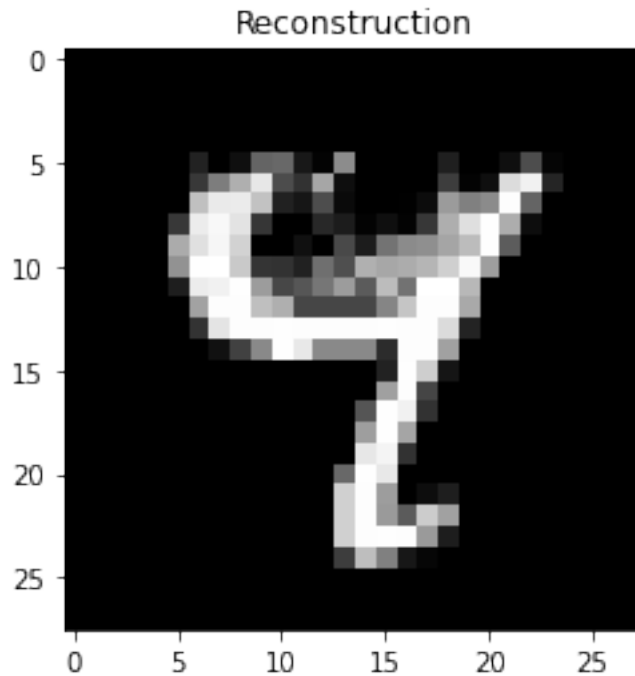
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Original Image



Corrupted Image





Problem 4

```
[1]: import numpy as np
      from matplotlib import pyplot as plt
      import math
```

```
[2]: N, p = 30, 20
      np.random.seed(0)
      X = np.random.randn(N,p)
      Y = 2*np.random.randint(2, size = N) - 1
      lamda = 30

      theta = 0.1 * np.random.randn(p)
      phi = 0.1 * np.random.randn(p)
      alpha = 3e-1
      beta = 1e-4

      epoch = 1000
      L_val = []
      d_phi_val = []
      d_theta_val = []
```

We want to apply Stochastic Gradient Descent; so we should decompose $L(\theta, \phi)$ into N different equations;

$L_i = \sum_{i=1}^N \log(1 + \exp(-Y_i(X_i - \phi)^T \theta)) - \frac{\lambda}{2} \|\phi\|^2$, so we can attain

$$\frac{\partial L_i}{\partial \theta} = \frac{-Y_i(X_i - \phi)}{1 + \exp(Y_i(X_i - \phi)^T \theta)} \text{ and } \frac{\partial L_i}{\partial \phi} = \frac{Y_i \theta}{1 + \exp(Y_i(X_i - \phi)^T \theta)} - \lambda \phi.$$

```
[20]: for _ in range(epoch):
        for __ in range(N):
            ##### implement SGD #####

            K = np.random.randint(N)

            # differential of L for phi
            d_phi = Y[K] / (1 + np.exp(Y[K] * ((X[K]-phi.reshape(1,-1)) @ theta))) *
            ↪ theta - lamda * phi

            # differential of L for theta

            # Actually, you can calculate inner product using * operator in numpy.
            d_theta = (-Y[K] * (X[K]-phi)) / (1 + np.exp(Y[K] * ((X[K]-phi.
            ↪ reshape(1,-1)) @ theta)))

            # You should train phi first!

            phi += (beta * d_phi)
            theta -= (alpha * d_theta)

            L_i = np.average(np.log(1 + np.exp(-Y * ((X - phi.reshape(1,-1)) @
            ↪ theta)))) - lamda/2 * np.linalg.norm(phi, axis=0, ord=2) **2
            d_phi = np.average(Y / (1 + np.exp(Y * ((X-phi.reshape(1,-1)) @ theta)))) *
            ↪ theta - lamda * phi
            d_theta = np.average((-Y / (1 + np.exp(Y * ((X-phi.reshape(1,-1)) @
            ↪ theta)))) .reshape(-1,1)*(X-phi.reshape(1,-1)), axis=0)

            L_val.append(L_i)
            d_phi_val.append(d_phi)
            d_theta_val.append(d_theta)
```

```
[21]: fig, ax = plt.subplots(figsize=(20, 20))
plt.subplots_adjust(left=0.125,
                    bottom=0.1,
                    right=0.9,
                    top=0.9,
                    wspace=0.2,
                    hspace=0.35)

plt.subplot(3, 1, 1)
plt.title("L function")
plt.plot(L_val)
plt.subplot(3, 1, 2)
plt.title("||D_theta L||")
```

```
plt.plot(np.linalg.norm(d_theta_val, axis=1, ord=2))
plt.subplot(3, 1, 3)
plt.title("||D_phi L||")
plt.plot(np.linalg.norm(d_phi_val, axis=1, ord=2))
plt.show()
```

